Measured quantities of the optimization problem

A* + nonlinear MPC navigation stack, Unitree G1

Auto-generated from `viz/out/results.json`

2026-08-24

What this document is. A staging file. Every number here is produced by `viz/make_results.py` from the same modules the deployed planner imports, and is meant to be moved into the report section named beside it. Nothing in it is written by hand, and it is not itself a chapter.

Contents

1 Measured quantities of the optimization problem

This file is generated by `viz/results_tex.py` from `viz/out/results.json`; every number below is computed by the same modules the deployed planner imports. Do not edit it by hand: re-run `python3 viz/make_results.py`. Each block carries a note naming the section of the report it is meant to feed; emptying `\resNote` removes all of them at once.

1.1 Provenance

All quantities in this document were produced on 2026-08-24from commit `a54dfe30ec` of branch `G1_optimal_trajectory`, with the deployed profile `planner_params_g1.yaml`and the recorded run `industrial_plant_v3`. The measurement stack is CasADi 3.7.2, NumPy 1.26.4, Python 3.10.12.

The configuration under test is $N = 15$, $\Delta t = 0.20$ s (a 3.0 s horizon), cruise speed $v_{\mathrm{ref}} = 0.20$ m/s, input envelope (0.30, 0.02, 0.30) in m/s and rad/s, barrier $(W_{\mathrm{obs}}, r_{\mathrm{obs}}) = (120,\ 0.40$ m), integrator `midpoint`, reference mode `time`, terminal constraint `none`.

1.2 Where each block belongs in the report

This table is the point of the file: it is the integration plan. Delete it once the blocks have been moved.

Table 1: Destination of each block of measurements. The report sections are named by their label, not by their number, because the numbering is still going to change.

block	content	target section in the report
§1.3	Model discretisation, truncation order	`sec:discretization`
§1.4	Open-loop prediction error	`sec:model, sec:mismatch`
§1.5	NLP size and sparsity	`sec:dims (tab:nlp)`
§1.6	Condensed against sparse parametrisation	`sec:cost`
§1.7	AD against finite differences	`sec:solver, sec:impl`
§1.8	Exact Hessian against L-BFGS	`sec:solver`
§1.9	Interior point against active set	`sec:solver`
§1.10	KKT, LICQ, second-order conditions	`new, next to sec:constraints`
§1.11	Exact ℓ^1 penalty	`new, next to sec:barrier`
§1.12	Terminal equilibrium constraint	`sec:terminal`
§1.13	Constraint tightening from measured error	`new, next to sec:barrier / sec:mismatch`
§1.14	Solution regularity, bifurcation	`sec:barriersweep`
§1.15	Path-parametrised reference	`sec:refwarm`
§1.16	Horizon and sampling time	`sec:horizon, sec:dtsweep`
§1.17	Control horizon, and why not move blocking	`new, next to sec:horizon`
§1.18	Multi-objective scalarisation	`sec:weights`

1.3 Model discretisation: truncation order

Feeds Report `sec:discretization`. Replaces the claim that the pose integration is "one forward-Euler step" with a measured convergence order.

The global integration error was fitted against the step size on three motion regimes. The measured orders are 1.00 for forward Euler and 2.00 for the mid-point rule (Table 2), i.e. exactly the first and second order predicted by the truncation analysis. At the deployed $\Delta t = 0.20$ s and over a 3 s window the two integrators differ by 1.74×10^{-2} m against 8.70×10^{-5} m, a factor 200.

Table 2: Fitted global truncation order of the two integrators, by motion regime (log–log fit of the global error against the step size).

motion regime	order, Euler	order, mid-point
nominal ($v_x = 0.2$, $\omega = 0.3$)	1.00	2.00
with lateral drift	1.00	2.00
fast rotation ($\omega = 1.0$)	1.00	2.00

Table 3: Global error against the step size, nominal ($v_x = 0.2$, $\omega = 0.3$) regime. The orders of Table 2 are the log–log slopes of these two columns.

Δt [s]	error, Euler [m]	error, mid-point [m]
0.2000	1.74×10^{-2}	8.70×10^{-5}
0.1000	8.70×10^{-3}	2.17×10^{-5}
0.0500	4.35×10^{-3}	5.44×10^{-6}
0.0250	2.17×10^{-3}	1.36×10^{-6}
0.0125	1.09×10^{-3}	3.40×10^{-7}

1.4 Open-loop prediction error along the horizon

Feeds Report `sec:model` *and* `sec:mismatch`*. This is the quantity that decides whether the integrator is worth improving, and it says it is not: read it immediately after § 1.3.*

Each MPC prediction recorded in the run was compared with the pose the robot actually reached $k\,\Delta t$ later, over 762 usable cycles. The residual at $k = 0$ is 0.0241 m and measures time alignment between the two series, not the model. Subtracting it, the prediction diverges by 0.305 m at the end of the horizon.

That divergence is 18 times the truncation error of forward Euler over the same window and 3505 times that of the mid-point rule (§ 1.3). The discretisation is therefore *not* the limiting term of the prediction: what the horizon loses comes from the plant, not from the integrator, and refining the scheme would buy nothing measurable in closed loop.

Table 4: Open-loop prediction error along the horizon, over 762 cycles of the recorded run `industrial_plant_v3`.

k	$k\,\Delta t$ [s]	median error [m]	95th pct. [m]
0	0.0	0.0241	0.0610
1	0.2	0.0295	0.0707
2	0.4	0.0443	0.0817
3	0.6	0.0425	0.1134
4	0.8	0.0529	0.1557
5	1.0	0.0754	0.1943
6	1.2	0.1032	0.2267
7	1.4	0.1273	0.2580
8	1.6	0.1521	0.2849
9	1.8	0.1762	0.3105
10	2.0	0.2027	0.3363
11	2.2	0.2302	0.3647
12	2.4	0.2561	0.3963
13	2.6	0.2819	0.4273
14	2.8	0.3064	0.4455
15	3.0	0.3291	0.4756

1.5 Size and sparsity of the nonlinear program

Feeds Report `sec:dims` *and replaces its Table* `tab:nlp`*, whose numbers are those of the Go2 profile ($N = 50$). The row in bold is the deployed configuration.*

At the deployed $N = 15$ the program carries 141 decision variables and 156 constraints, of which 96 are equalities (the dynamics plus the initial condition) and 60 are simple bounds on the inputs; 121 quantities enter as CasADi parameters, so the expression graph is built once and only numbers are written into it between cycles.

The constraint Jacobian is 1.73% dense and the Hessian of the Lagrangian 4.45% (Table 5). Both densities fall as N grows while the nonzero counts grow linearly, which is the signature of the multiple-shooting parametrisation: no constraint couples distant stages, so the cost of one interior-point iteration is linear in the horizon. § 1.6 builds the same program in the condensed parametrisation and measures what that trade actually is, rather than asserting it.

Table 5: Size and sparsity of the NLP against the prediction horizon, from the deployed profile. Bold: the deployed configuration.

N	vars	eq.	bounds	params	jac nnz	jac dens. [%]	hess nnz	hess dens. [%]
10	96	66	40	91	256	2.52	600	6.51
15	**141**	**96**	**60**	**121**	**381**	**1.73**	**885**	**4.45**
25	231	156	100	181	631	1.07	1455	2.73
50	456	306	200	331	1256	0.54	2880	1.39

1.6 Condensed against sparse parametrisation of the same program

Feeds Report `sec:cost`*, which asserts multiple shooting is the right choice "because a condensed single-shooting formulation would produce a small but dense problem with a much worse conditioned Hessian". Half of that is now measured and half of it is not: this block reports which half.*

The same optimal control problem was built in both parametrisations, with the transition map written once and shared, so that what is compared is the parametrisation and not two different models. Eliminating the states by recursive substitution removes the dynamic equalities altogether: at $N = 15$ the program shrinks from 141 variables and 156 constraints to 45 and 60.

The structural half of the claim holds exactly, and the Hessian is where it shows: 4.45% dense in the sparse parametrisation against 100.00% in the condensed one, which is a full matrix. Condensing trades a large banded problem for a small dense one, exactly as stated.

The performance half does not follow from that. On this run the sparse parametrisation is faster at $N = 5$ and the condensed one elsewhere (Table 6), and each entry is a single cold solve, so the timings carry the variance of one sample while the dimensions and the densities beside them are exact. What can be asserted without a stopwatch is the conditioning argument: the condensed form integrates the model in open loop over the whole horizon, so the error compounds step by step and the problem degrades with N and with any instability of the plant. On a kinematic, stable model that defect does not surface — which is precisely why this comparison cannot be used to argue the general case, and why the report should claim the structure rather than the speed.

1.7 Derivatives: algorithmic differentiation against finite differences

Feeds Report `sec:solver` / `sec:impl`*, which currently assert that "exact derivatives are available from CasADi's AD" without measuring the alternative.*

Table 6: Multiple (M) against single (S) shooting on the same instance. Timings are single cold solves; dimensions and densities are exact.

N	vars M / S	cons M / S	jac dens. [%] M / S	hess dens. [%] M / S	solve [ms] M / S
5	51 / 15	56 / 20	4.59/6.67	12.11/100.00	86 / 147
15	141 / 45	156 / 60	1.73/2.22	4.45/100.00	209 / 191

At a representative solve the objective has 141 variables. One evaluation of the objective costs 105.0 μs and one full gradient by reverse-mode algorithmic differentiation 145.4 μs, i.e. 1.51 objective evaluations (median of repeated pairs, range 1.28–1.78), independently of the number of variables. The same gradient by finite differences costs 142 evaluations one-sided and 282 central, and is less accurate at every step size: the best relative error reachable is 5.50×10^{-8} one-sided and 1.29×10^{-10} central (Table 7), against machine precision for AD.

The cost is the argument that closes the question for a real-time loop: central differences alone would consume 24% of the 125 ms cycle budget, for a gradient that is worse. The ratio itself is a micro-benchmark on ~ 100 μs timings and its second digit is not meaningful; what is stable, and is the point, is that it stays a small constant.

Table 7: Accuracy of the finite-difference gradient against the step size, measured against the AD gradient. The optimum sits at $h = 1.49 \times 10^{-8}$ one-sided (theory: $\sqrt{\varepsilon} = 1.49 \times 10^{-8}$) and $h = 6.06 \times 10^{-6}$ central (theory: $\varepsilon^{1/3} = 6.06 \times 10^{-6}$).

step h	rel. error, one-sided	rel. error, central
1.00×10^{-4}	2.48×10^{-4}	2.80×10^{-8}
6.06×10^{-6}	1.50×10^{-5}	1.29×10^{-10}
1.00×10^{-6}	2.48×10^{-6}	5.43×10^{-10}
1.49×10^{-8}	5.50×10^{-8}	3.45×10^{-8}
1.00×10^{-10}	8.03×10^{-6}	4.31×10^{-6}

1.8 Exact Hessian against a quasi-Newton approximation

Feeds Report `sec:solver`: it is the second half of the AD argument — AD is what makes the exact Hessian affordable, and this is what the exact Hessian buys.

Solving the same instance with the exact Hessian of the Lagrangian takes 20 iterations against 36 with the limited-memory quasi-Newton approximation, a 44% reduction, and both converge to the same objective value (Table 8). Since CasADi supplies the exact second derivatives at a cost comparable to the first, the full Newton step is the cheaper option here, not the more expensive one.

Table 8: Interior-point iterations with the exact Hessian and with the limited-memory quasi-Newton approximation, on the same instance. Bold: the lower iteration count.

Hessian	iterations	solve [ms]	$J^\star$	status
exact Hessian	**20**	210.9	9244	`Solve_Succeeded`
L-BFGS	36	297.1	9244	`Solve_Succeeded`

1.9 Interior point against active set

Feeds Report `sec:solver`, which argues for an interior-point method on the grounds that the constraint set is dominated by equalities. That argument is now testable rather than asserted, because the obstacle formulation is switchable and the same system can be put in both regimes.

The rule of thumb is that active-set methods win when the inequalities are few and interior-point methods win when they are many. This stack can be moved between the two regimes without changing anything else: with the obstacles in the objective the program carries 60 inequalities, and with the obstacles as genuine constraints it carries 300.

The rule does not reproduce (Table 9): the active-set solver does not win even in the small regime, where the interior-point method is already 33.5× faster. The *direction* is confirmed — the margin widens to 10.0× when the inequalities multiply — so the break-even point, if there is one, lies below the smallest regime this problem can be put in.

Two cautions, without which the numbers would be misleading. First, the real advantage of an active set in MPC is warm starting *between consecutive solves*: the active set changes by a few rows per cycle and the factorisation is reused. Both solvers are started cold here, on purpose, so as to favour neither — which removes from the active-set method exactly what makes it competitive in a receding-horizon loop. Second, the rule of thumb is stated for convex programs, and this one is not.

A by-product worth reporting: with the exact Hessian of the Lagrangian the QP subproblem is repeatedly flagged indefinite. That is expected and instructive — a non-convex program can produce a non-convex QP, which has no unique solution — and it is the reason a Gauss-Newton Hessian, positive semi-definite by construction, is the standard recommendation for SQP.

At least one pair of solves converged to different objective values. Comparing the time of two solves that ended in different minima means nothing; that row is not evidence for either method.

Table 9: The same instance solved by a primal–dual interior-point method and by an SQP with an active-set QP solver, in both obstacle regimes. Both are started cold. Bold: the faster of the two.

regime	ineq.	IPOPT ms / iter	SQP ms / iter	speed-up	same min.
penalty (obstacles in the cost)	60	**49** / 21	1656 / -1	33.5×	no
penalty (obstacles in the cost)	60	**34** / 15	138 / 6	4.0×	yes
penalty (obstacles in the cost)	60	**47** / 16	348 / 6	7.5×	yes
ℓ^1 (obstacles as constraints)	300	**76** / 21	5216 / 11	68.7×	no
ℓ^1 (obstacles as constraints)	300	**112** / 21	5000 / 7	44.7×	yes
ℓ^1 (obstacles as constraints)	300	**45** / 12	453 / 1	10.0×	no

1.10 Optimality conditions along the mission

New material: the report has no KKT section at all. It belongs next to `sec:constraints`*, and it is what licenses calling $z^\star$ an optimum rather than "what IPOPT returned".*

The first- and second-order conditions were checked at 9 cycles sampled along a recorded mission. LICQ holds at every one of them (*yes*), strict complementarity holds at every one (*yes*), and the reduced Hessian is positive definite on the critical cone at every one (*yes*), the smallest projected eigenvalue over the profile being 8.37×10^{-1}. The solution is therefore a strict local minimum satisfying the second-order sufficient conditions, and the multipliers are unique.

The structural remark first: the obstacle terms live in the objective, not in the constraints, so there is no obstacle multiplier and no non-trivial active-set combinatorics to resolve. What remains active is the dynamics plus the input bounds, and the bounds are active up to 41 times per cycle.

The quantity worth reporting is what that does to the critical cone, whose dimension falls from 45 at the first sampled cycle to 5 at the last: with 141 variables and 136 active constraints, only 5 degree(s) of freedom remain. Over the mission the controller moves from *cost-driven* to *constraint-driven*: towards the end it no longer chooses the trajectory so much as undergo it. The input envelope is what does this — a lateral bound of 0.02 m/s does not *limit* a degree of freedom, it *removes* it. The same asymmetry bounds what any state constraint can ask of this robot: it can

advance along its own heading, but it cannot translate away from a wall, which is the limit § 1.13 runs into from the other side.

The cone dimension over the sampled profile ranges between 4 and 45 (Table 10), and the identity $\dim(\text{cone}) = n_{\text{var}} - \#\text{active holds}$ at every cycle, which is what strict complementarity buys.

Table 10: Optimality diagnostics at sampled cycles of the recorded run `industrial_plant_v3`. "active" counts equalities and active bounds together; "rank" is the rank of the Jacobian of the active constraints, so LICQ holds when the two coincide.

cycle	t [s]	active	of which bounds	rank	LICQ	cone dim.	$\lambda_{\min}$ proj.
0	0	96	0	96	yes	45	8.78×10^{-1}
95	60	98	2	98	yes	43	8.75×10^{-1}
190	105	100	4	100	yes	41	8.37×10^{-1}
285	153	97	1	97	yes	44	8.78×10^{-1}
380	197	96	0	96	yes	45	8.78×10^{-1}
475	242	105	9	105	yes	36	9.16×10^{-1}
570	294	99	3	99	yes	42	8.83×10^{-1}
665	359	137	41	137	yes	4	7.19×10^{0}
761	430	136	40	136	yes	5	1.57×10^{0}

1.11 Soft obstacle constraint: exact ℓ^1 penalty against ℓ^2

New material, and the strongest single addition available to the report: it turns `sec:barrier`'s admission that "the barrier is not an exact penalty, so a finite weight admits a finite violation" from a caveat into a measurement, with the threshold at which the violation is exactly zero.

The obstacle term was re-posed as a genuine inequality constraint $d(x_k, \mathcal{P}) \geq d_{\text{safe}}$ with $d_{\text{safe}} = 1.10$ m, relaxed by a slack variable penalised either linearly (ℓ^1) or quadratically (ℓ^2) with weight ρ_s. Solved as a hard constraint the instance is feasible and the largest multiplier of an active distance constraint is $\mu^\star = 8905$.

The two penalties then behave exactly as the exact-penalty theorem predicts (Table 11). The ℓ^1 slack is a threshold phenomenon: it is nonzero below the threshold and drops to zero — not small, zero to solver tolerance — from $\rho_s = 1 \times 10^4$ onwards, i.e. once ρ_s exceeds the multiplier of the corresponding hard constraint. The ℓ^2 slack instead decays like $1/\rho_s$, the fitted log–log slope of its tail being -1.00 against a predicted -1, and never reaches zero at any finite weight.

The practical reading for this stack is that a soft constraint can be made *exactly* satisfied at a finite, computable weight, provided the penalty is non-smooth at the origin; the smooth quadratic relaxation that is more comfortable for the solver is precisely the one that can never close the violation.

Table 11: Residual constraint violation against the penalty weight, for the non-smooth and the smooth relaxation of the same distance constraint. Bold: the violations that are exactly zero, which is the result the exact-penalty theorem predicts and the smooth relaxation never attains. Zero entries are below the solver tolerance of 1×10^{-8} m.

ρ_s	max slack, ℓ^1 [m]	max slack, ℓ^2 [m]
1×10^3	1.46×10^{-1}	2.61×10^{-1}
1×10^4	**0**	1.30×10^{-1}
1×10^5	**0**	3.98×10^{-2}
1×10^6	**0**	4.38×10^{-3}
1×10^7	**0**	4.42×10^{-4}
1×10^8	**0**	4.42×10^{-5}

1.12 Adding a terminal equilibrium constraint

Feeds Report `sec:terminal`*, which states that the formulation carries no terminal ingredient and that none of the standard guarantees apply. This measures what supplying one would actually cost.*

The program was re-solved with a terminal equilibrium constraint — the velocity states driven to zero at the last node, relaxed by a slack so that the comparison can never be decided by infeasibility. Across the sampled cycles the terminal slack never leaves zero (maximum 0, always admissible: *yes*): the constraint is reachable at every operating point tested, so the terminal set is not empty in practice and recursive feasibility is not obtained at the price of an infeasible program.

The cost of imposing it, measured as the relative increase of the optimal objective, ranges from 3.3% to 85.2% depending on the cycle. The upper end is paid where the horizon was being used to keep moving, which is the expected trade: a terminal stop constraint buys the stability argument by spending part of the horizon on braking.

That the slack is always zero is not by itself evidence that the constraint works — an unimplemented constraint would report the same thing — and the reason it is zero has to be stated, because it is a property of this profile and not of the method. The discrete lag is $1 - e^{-\Delta t/\tau} = 1.000000$ at $\tau = 0.00100$ s against $\Delta t = 0.20$ s, i.e. degenerate: the model reduces to $v_{k+1} = u_k$ and the robot reaches zero velocity in a single step, so the terminal set is trivially reachable from anywhere. On hardware, with an identified actuator time constant, the slack becomes the quantity that decides the maximum speed from which the horizon can still bring the robot to a stop — which is the physical reading of the terminal feasible set, and the form in which this constraint would actually bind.

1.13 Constraint tightening from the measured prediction error

New material. Feeds Report `sec:barrier` *and* `sec:mismatch`*: the clearance constraint is imposed on the* predicted *trajectory, and § 1.4 measures how far that diverges from the executed one. This closes the gap instead of noting it.*

The obstacle constraint is tightened by a margin $\gamma(k)$ that grows along the horizon, $\|p_k - o_j\| \geq d_{\text{safe}} + \gamma(k) - s_{jk}$. The margin is not postulated: it is read off the 95th percentile of the prediction error recorded in the run of § 1.4, so the tube is derived from data rather than from an assumption about the disturbance.

Three properties hold by construction and are worth checking rather than assuming (Table 12). $\gamma(0) = 0.0000$ exactly: at the first node the state is fixed by an equality constraint, so there is nothing to hedge against and the constraint is not tightened where it would only remove feasible motion. The margin is monotone (*yes*), as uncertainty is. And it reaches 0.4146 m at the end of the horizon, which is the same order as the safety radius itself — the tube is not a decoration.

Measured on the predicted trajectory (6 cases, Table 13), the outcomes separate cleanly and each is informative: where $d_{\text{safe}} + \gamma$ stays below the clearance the trajectory already keeps, the constraint is inactive and correctly does nothing; where it bites, the predicted clearance increases with the slack still exactly zero, so the margin is *respected* rather than violated and paid for; and where it asks for more than the input set can deliver, the ℓ^1 penalty of § 1.11 yields instead of rendering the program infeasible — which is precisely why that relaxation was chosen.

Limit of this measurement, to be stated in the report. *The effect is not observable in the closed-loop harness: the executed clearance comes out identical with and without tightening, because the loop is closed by tracking a look-ahead setpoint on the predicted trajectory with a proportional controller, which washes out fine differences between MPC solutions. The tightening guarantees the margin* in the plan*, and that is where it has been verified.*

Table 12: Constraint back-off derived from the 95th percentile of the measured prediction error. The offset at $k = 0$ is subtracted: it is time misalignment, not model uncertainty, and including it would inflate the tube by a constant.

k	$k\,\Delta t$ [s]	$\gamma(k)$ [m]
0	0.0	0.0000
1	0.2	0.0097
2	0.4	0.0207
3	0.6	0.0524
4	0.8	0.0948
5	1.0	0.1334
6	1.2	0.1658
7	1.4	0.1970
8	1.6	0.2239
9	1.8	0.2495
10	2.0	0.2753
11	2.2	0.3037
12	2.4	0.3353
13	2.6	0.3663
14	2.8	0.3845
15	3.0	0.4146

Table 13: Effect of the tightening on the predicted trajectory.

scenario	d_{safe}	clear. without	clear. with	Δ	slack w/o / with	outcome
narrow_gap	0.40	0.7500	0.8146	0.0646	0 / 0	tightening effective
narrow_gap	0.70	0.7500	1.1146	0.3646	0 / 0	tightening effective
narrow_gap	1.00	1.0000	1.2816	0.2816	0 / 0.133	infeasible
u_trap	0.40	1.3417	1.3417	0.0000	0 / 0	constraint inactive
u_trap	0.70	1.3417	1.3417	0.0000	0 / 0	constraint inactive
u_trap	1.00	1.3417	1.4146	0.0729	0 / 0	tightening effective

1.14 Regularity of the solution: the left–right bifurcation

Feeds Report `sec:barriersweep`. *The report already shows that the barrier weight governs a cliff; this gives the cliff its mechanism, and it is the one place where the non-convexity of the program becomes visible as a discontinuity of the control law rather than as an iteration count.*

With an obstacle centred on the reference, the program admits two symmetric local minima — pass left, pass right — and the solution as a function of the barrier weight is regular only as long as one of them dominates. Solving from a left-biased and a right-biased initial guess and measuring the separation between the two returned trajectories locates the transition between $W_{\text{obs}} = 200$ and $W_{\text{obs}} = 300$ (Table 14). Below it the two guesses collapse onto the same trajectory; above it they do not, and the optimizer's choice becomes a discontinuous function of the current state.

The deployed weight sits below the threshold (*yes*), and on the hardest cycle of the recorded run — cycle 664, the one with the most obstacles inside the search radius — the sweep never bifurcates (*yes*). The design is therefore on the regular side of the transition rather than accidentally past it, which is the statement the report needs in order to treat the MPC law as well defined.

Two readings follow, and both are actionable. First, past the threshold the two minima are *not* equivalent: their objective values differ by up to 0.55%, so the initial guess decides not merely which side the robot passes on but how much the manoeuvre costs. Second, since the deployed weights are on the regular side, the cost-spike guard that clears the warm-start cache when the objective jumps is protecting against a phenomenon that does not occur at these weights — worth stating as a measured fact about the deployed configuration rather than removing on the strength of one sweep.

Table 14: Left-biased against right-biased solve of the same instance, against the barrier weight. The separation is the distance between the two returned trajectories: a value at solver tolerance means the two guesses converged to the same minimum.

W_{obs}	separation [m]	$J^\star$ left	$J^\star$ right	iter. left	iter. right
60.0	1.16×10^{-6}	4953	4953	21	21
120	6.34×10^{-9}	9244	9244	19	19
200	7.11×10^{-10}	14252	14252	24	23
300	3.69×10^{0}	20013	20122	25	29
450	3.89×10^{0}	28152	28297	25	23
600	3.98×10^{0}	36019	36187	27	28
900	4.22×10^{0}	51333	51596	29	25
1400	4.49×10^{0}	76236	76606	24	26

1.15 Reference generation: time-parametrised against path-parametrised

Feeds Report `sec:refwarm`, *which defines the reference by advancing along the path at a fixed cruise speed. This measures what that fixed speed costs.*

The deployed reference samples the smoothed path at a constant cruise speed $v_{\text{ref}} = 0.20$ m/s, which is set below $v_{x,\max}$ on purpose — with $v_{\text{ref}} = v_{x,\max}$ the tracker saturates permanently and never settles onto the path. The price is that 33% of the available forward speed is unreachable by construction: no cost weight can recover it, because the reference itself never asks for it.

Re-posing the same program with the path abscissa as a decision variable — the tracker chooses how far to advance rather than being told — removes the constant and lets the bound do the limiting. Over 6 cycles replayed from the recorded run, the mean commanded speed rises from 0.189 to 0.300 m/s and the distance covered within one horizon from 0.568 to 0.894 m, a 58% gain (Table 15).

It is not free: the iteration count rises from 10.7 to 19.7, because the extra decision variable removes the term that was pinning the solution along the path. Reported as a trade rather than as an improvement, this is the cleanest reformulation available to the stack, and the one that would let v_{ref} disappear from the parameter file.

Table 15: Time-parametrised against path-parametrised reference, averaged over 6 cycles replayed from the recorded run `industrial_plant_v3`.

quantity	time-parametrised	path-parametrised
mean v_x [m/s]	0.189	0.300
advance over the horizon [m]	0.568	0.894
IPOPT iterations	10.7	19.7

1.16 Prediction horizon and sampling time

Feeds Report `sec:horizon` and `sec:dtsweep`, whose tables are the Go2 ones. Note that here N and Δt are swept independently rather than at a fixed look-ahead, so the two report tables collapse into one.

The two parameters are not interchangeable: the product $N\Delta t$ sets how far the controller sees, N alone sets how much the solve costs, and Δt alone sets how faithful the prediction is. They were therefore swept jointly over 40 configurations, in closed loop, against a cycle budget of 125 ms; 3 of them exceed it at the 95th percentile of the per-cycle solve time.

The headline is counter-intuitive and worth stating first: *lengthening the horizon makes the closed loop worse.* Split at $N\Delta t = 6$ s, the short-horizon group reaches the goal in 11.0 s with a worst-case clearance of 0.000 m, against 22.7 s and 0.009 m for the long-horizon group — and the clearance axis does not discriminate: both groups come within a couple of centimetres of an obstacle, so on these scenarios neither horizon is being solved safely and only the time is informative. The mechanism is the same one that produces the livelock elsewhere in the stack: the reference extends over a path the discrete planner will replan anyway, so a longer horizon commits the controller to tracking a target that is already due to change. That explanation has a competitor — simply too many decision variables — and § 1.17 is the experiment that separates the two; it reports there which one this data supports.

Ranking the aggregated configurations on the three objectives that matter —time to goal, worst-case clearance and tail solve time— leaves 6 non-dominated points, and the deployed $N = 15$, $\Delta t = 0.20$ s is *dominated*: another configuration matches or beats it on all three. The cheapest point of the non-dominated set is $N = 5$, $\Delta t = 0.40$ s, at 23.6 ms of 95th-percentile solve time against 60.3 ms for the deployed one.

Before changing the deployed profile. *These are synthetic scenarios with static obstacles and frequent replanning. A very short horizon holds up only because the discrete planner is doing the avoidance; with dynamic obstacles, or a slower planner, the margin would disappear. The result is a reason to run the comparison on real missions, not a reason to retune from a table.*

1.17 Control horizon: separating preview from degrees of freedom

New material, and it answers a question § 1.16 cannot: there N governs both how far the controller looks and how many free inputs it has, so a degradation with N has two candidate causes. Belongs next to `sec:horizon`.

Freeing only the first N_c inputs and holding the rest at the last free value decouples the two roles of the horizon: the prediction still runs to N, but the program carries fewer variables. Over 10 configurations (Table 18) the two effects separate cleanly.

Table 16: Horizon campaign, scenario `narrow_gap`. Bold: the deployed configuration $N = 15$, $\Delta t = 0.20$ s.

N	Δt [s]	T [s]	vars	median [ms]	p95 [ms]	goal	TTG [s]	min clear. [m]
5	0.10	0.5	51	11.2	26.1	yes	9.1	0.450
5	0.20	1.0	51	9.3	24.7	yes	9.4	0.450
5	0.30	1.5	51	10.9	27.3	yes	9.3	0.451
5	0.40	2.0	51	10.5	22.7	yes	9.6	0.450
10	0.10	1.0	96	15.9	27.6	yes	9.1	0.450
10	0.20	2.0	96	16.9	47.6	yes	9.4	0.450
10	0.30	3.0	96	15.4	45.9	yes	9.3	0.451
10	0.40	4.0	96	17.5	47.7	yes	9.6	0.450
15	0.10	1.5	141	19.8	67.8	yes	9.1	0.450
15	**0.20**	**3.0**	**141**	**20.7**	**60.3**	**yes**	**9.4**	**0.450**
15	0.30	4.5	141	24.3	64.9	yes	9.3	0.451
15	0.40	6.0	141	27.5	95.3	yes	21.6	0.017
25	0.10	2.5	231	38.7	129.2	yes	9.1	0.450
25	0.20	5.0	231	39.4	65.1	yes	9.2	0.450
25	0.30	7.5	231	46.1	78.1	no	—	0.044
25	0.40	10.0	231	37.1	65.1	yes	23.2	0.305
40	0.10	4.0	366	42.0	173.1	yes	9.1	0.450
40	0.20	8.0	366	50.2	90.3	yes	24.2	0.214
40	0.30	12.0	366	61.4	111.0	yes	26.7	0.232
40	0.40	16.0	366	60.0	86.7	yes	23.2	0.296

Table 17: Horizon campaign, scenario `u_trap`. Bold: the deployed configuration $N = 15$, $\Delta t = 0.20$ s.

N	Δt [s]	T [s]	vars	median [ms]	p95 [ms]	goal	TTG [s]	min clear. [m]
5	0.10	0.5	51	11.9	30.1	yes	12.4	0.000
5	0.20	1.0	51	12.3	25.4	yes	12.8	0.000
5	0.30	1.5	51	11.2	20.1	yes	12.6	0.030
5	0.40	2.0	51	13.1	23.6	yes	12.8	0.000
10	0.10	1.0	96	13.2	26.7	yes	12.4	0.000
10	0.20	2.0	96	14.5	23.0	yes	12.8	0.000
10	0.30	3.0	96	15.9	35.3	yes	12.6	0.030
10	0.40	4.0	96	17.0	44.0	yes	12.8	0.000
15	0.10	1.5	141	18.5	39.5	yes	12.4	0.000
15	**0.20**	**3.0**	**141**	**19.2**	**36.8**	**yes**	**12.8**	**0.000**
15	0.30	4.5	141	21.7	48.0	yes	12.6	0.030
15	0.40	6.0	141	22.5	36.1	yes	18.8	0.041
25	0.10	2.5	231	28.1	52.4	yes	12.4	0.000
25	0.20	5.0	231	33.9	143.4	yes	13.2	0.002
25	0.30	7.5	231	37.4	101.2	yes	18.6	0.089
25	0.40	10.0	231	56.2	121.6	yes	19.2	0.229
40	0.10	4.0	366	42.9	92.5	yes	12.4	0.000
40	0.20	8.0	366	45.8	85.4	yes	26.4	0.009
40	0.30	12.0	366	59.0	104.5	yes	23.7	0.051
40	0.40	16.0	366	44.1	97.2	yes	20.4	0.256

Where the prediction horizon is already the right length, cutting the degrees of freedom is free: in 3 of the paired cases the time to goal and the minimum clearance are unchanged while the 95th percentile of the solve time falls, by up to 2.1× at $N = 15$. This is the input parametrisation argument in its cheapest form: the degrees of freedom past the first few are not what the closed loop was using.

The complementary question — whether the degradation a long horizon causes in § 1.16 comes from the preview or from the extra degrees of freedom — is *not* answered by this sweep: none of the horizons tested here degrades the closed loop, so there is no degradation to attribute. Answering it requires re-running this comparison over the horizons that do degrade, which is the cheapest missing measurement in this document.

On move blocking. *Holding the input constant over blocks of increasing length is the standard way to recover computation without shortening the horizon, and it is deliberately not used here. Wherever § 1.16 finds the useful horizon to be short, compressing a handful of variables into blocks changes nothing measurable; the control horizon above is the degenerate case of move blocking — one free block followed by one long one — and already delivers the saving; and computation is not the binding resource, since the deployed configuration sits well inside its cycle budget while the prediction error of § 1.4 does not. It is a considered choice, not an omission, and it would become the right technique if a rigorous terminal ingredient forced a long horizon.*

One implementation detail carries theory. Imposing the input bounds on all N steps when the inputs past N_c are the same repeated expression generates duplicate rows with identical gradients; if active, they violate LICQ and make the multipliers non-unique — breaking exactly the analysis of § 1.10. The bounds must be imposed on the free columns only.

One reading of the table is not about N_c at all and should not be passed over: in `u_trap` the minimum clearance is zero to display precision in every configuration, so the robot grazes an obstacle regardless of the control horizon. Whatever that scenario is testing, it is not being solved safely, and the comparison above is made between configurations that all touch.

Table 18: Closed-loop outcome against the control horizon at fixed prediction horizon.

scenario	N	N_c	vars	goal	TTG [s]	min clear. [m]	p95 [ms]
narrow_gap	5	1	39	yes	9.4	0.450	21.8
narrow_gap	5	5	51	yes	9.4	0.450	12.2
narrow_gap	15	1	99	yes	9.4	0.450	18.7
narrow_gap	15	5	111	yes	9.4	0.450	48.6
narrow_gap	15	15	141	yes	9.4	0.450	40.2
u_trap	5	1	39	yes	12.8	0.000	6.3
u_trap	5	5	51	yes	12.8	0.000	11.4
u_trap	15	1	99	yes	12.8	0.000	11.2
u_trap	15	5	111	yes	12.8	0.000	25.7
u_trap	15	15	141	yes	12.8	0.000	23.8

1.18 Multi-objective scalarisation and the Pareto front

Feeds Report `sec:weights`, which already exhibits one two-objective trade-off (effort against accuracy) but selects its operating point implicitly. This makes the scalarisation explicit and the front measurable.

The cost weights implement a scalarisation of three competing objectives — tracking accuracy, control effort and time to goal — with fixed coefficients. Sweeping the coefficients over the simplex and running each resulting controller in closed loop gives 21 points, of which 2 are non-dominated (Table 19); the front is convex: *yes*, so the weighted-sum scalarisation can in principle reach every point of it.

The relative excursion over the simplex is what says whether a trade-off exists at all: accuracy

moves by 50.6%, effort by 7.4% and time to goal by 4.3%. Only the first responds appreciably to the weights; the other two are very nearly fixed, because the speed along the path is decided by the kinematics and the input bounds rather than by the tuning.

The barycentre of the simplex, which would reproduce the tuning already in use, is not among the sampled points at this resolution, so the sweep cannot say whether the current weights are dominated. Sampling it explicitly is the cheapest way to make this section answer the question the report will ask of it.

Table 19: Closed-loop outcome of each scalarisation of the three objectives. Bold: non-dominated points.

κ	accuracy [m]	effort	time [s]	clearance [m]	non-dom.
(0.0, 0.0, 1.0)	**0.0203**	**0.0904**	**9.0**	**0.450**	yes
(0.0, 0.2, 0.8)	0.0208	0.0870	9.4	0.450	no
(0.0, 0.4, 0.6)	0.0214	0.0870	9.4	0.450	no
(0.0, 0.6, 0.4)	0.0215	0.0870	9.4	0.450	no
(0.0, 0.8, 0.2)	0.0217	0.0870	9.4	0.450	no
(0.0, 1.0, 0.0)	0.0217	0.0870	9.4	0.450	no
(0.2, 0.0, 0.8)	0.0190	0.0865	9.4	0.450	no
(0.2, 0.2, 0.6)	0.0293	0.0930	9.2	0.450	no
(0.2, 0.4, 0.4)	0.0197	0.0867	9.4	0.450	no
(0.2, 0.6, 0.2)	0.0197	0.0867	9.4	0.450	no
(0.2, 0.8, 0.0)	0.0190	0.0865	9.4	0.450	no
(0.4, 0.0, 0.6)	0.0197	0.0867	9.4	0.450	no
(0.4, 0.2, 0.4)	0.0197	0.0867	9.4	0.450	no
(0.4, 0.4, 0.2)	0.0197	0.0867	9.4	0.450	no
(0.4, 0.6, 0.0)	0.0216	0.0928	9.4	0.450	no
(0.6, 0.0, 0.4)	0.0189	0.0865	9.4	0.450	no
(0.6, 0.2, 0.2)	0.0189	0.0865	9.4	0.450	no
(0.6, 0.4, 0.0)	0.0189	0.0865	9.4	0.450	no
(0.8, 0.0, 0.2)	0.0198	0.0867	9.4	0.450	no
(0.8, 0.2, 0.0)	0.0198	0.0867	9.4	0.450	no
(1.0, 0.0, 0.0)	**0.0189**	**0.0865**	**9.4**	**0.450**	yes